CS3223 Project: SimpleDB+ Join Experiments

Joel Kalaivanan (A0291976J)
Jin Xunze (A0255453A)
Wang Xuxin (A0252982W)
23 October 2026

Index join was the fastest join algorithm in both experiments: 8 ms for the 2-table join and 41 ms for the 4-table join. Sort-merge join was the slowest, at 805 ms for the 2-table join.

1. Setup

All timings were measured in Eclipse on the SimpleDB+ engine with its default configuration: 8 buffers of 400 bytes and the heuristic query planner.

Database. The provided student database has only 9 students, too few to tell the algorithms apart. We used a larger copy of the same schema, generated by `CreateBigStudentDB` with a fixed random seed.

Table 1: Test database

Table	Rows	Blocks	Index
STUDENT	600	47	hash on MajorId
ENROLL	900	50	B-tree on StudentId
COURSE	90	9	none
DEPT	30	2	none

Restricting join methods. `JoinStrategy` limits the optimizer to one join algorithm: index, sort-merge, hash (partition-based) or nested loops. Where the allowed algorithm cannot be used for a join, the optimizer falls back to nested loops.

Timing. `JoinExperiments` runs each configuration once to warm up, then 5 timed runs, each planning the query and reading every result row. We report the median.

2. Experiment 1: 2-Table Join

Index join ran the 2-table join in 8 ms and sort-merge in 805 ms. Every algorithm returned the same 900 rows.

```
select sname, grade from student, enroll where sid = studentid
```

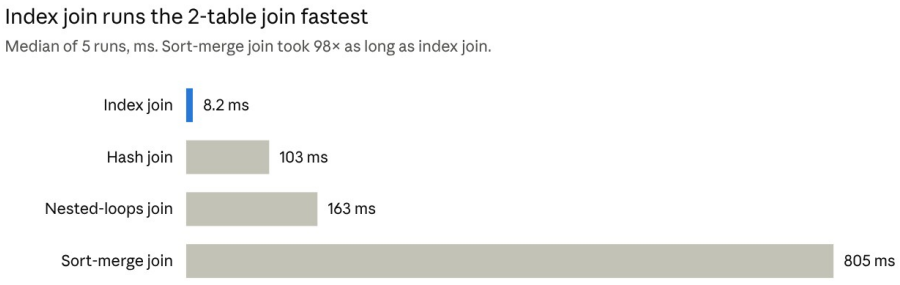


Figure 1: Median running time of the 2-table join by join algorithm (5 runs after 1 warm-up)

Table 2: Plans used in Experiment 1

Join algorithm	Plan used
Index	For each STUDENT row, look up ENROLL through the B-tree index on StudentId
Sort-merge	Sort STUDENT by SId and ENROLL by StudentId, then merge
Hash	Partition both tables on the join field, then join each partition pair
Nested loops	Scan all of ENROLL once per STUDENT row

3. Experiment 2: 4-Table Join

Of the three plans, Plan A (index joins) took 41 ms, Plan B (hash) 316 ms and Plan C (nested loops) 727 ms. Every plan returned the same 2,805 rows. The big database has no SECTION table, so the join links ENROLL, STUDENT, DEPT and COURSE.

```
select sname, dname, title, grade from student, enroll, dept, course
where sid = studentid and majorid = did and did = deptid
```

Each plan comes from restricting the optimizer to one join algorithm. The restriction changed the join order as well as the algorithm.

Table 3: Plans used in Experiment 2 (⋈ = join)

Plan	Join methods allowed	Join order and algorithms
A	Index only	((DEPT ⋈ STUDENT by hash index on MajorId) ⋈ ENROLL by B-tree index on StudentId) ⋈ COURSE by nested loops
B	Hash only	((DEPT ⋈ COURSE) ⋈ STUDENT) ⋈ ENROLL, all hash joins
C	Nested loops only	((DEPT ⋈ COURSE) ⋈ STUDENT) ⋈ ENROLL, all nested loops

Plan A has no index for the COURSE join (DeptId is not indexed), so the optimizer fell back to nested loops for that step.

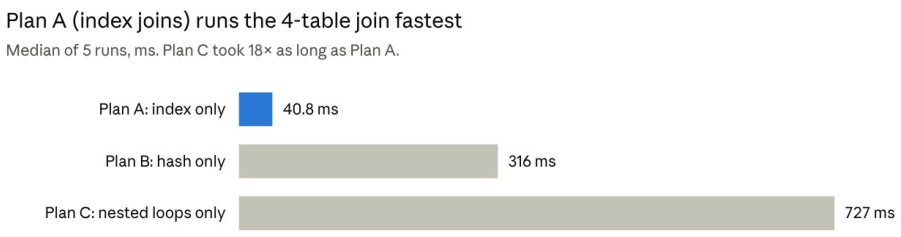


Figure 2: Median running time of the 4-table join by plan (5 runs after 1 warm-up)

4. Discussion

The ranking was consistent: index join was fastest in both experiments and hash join beat nested loops in both. Sort-merge, measured in Experiment 1, was slowest.

Why sort-merge is slow. Sorting first splits each input into many short sorted runs, each written to its own temporary table, then merges them two at a time until one run is left. With 8 buffers every merge pass writes and re-reads all records through new temporary files, which costs far more than the merge itself.

Why index join is fastest. Each STUDENT row needs one index lookup, which matches on average 1.5 ENROLL rows. The index pages are small and reused by every lookup, so they stay cached, and no temporary files are written.

Hash against nested loops. Hash join reads each input once and writes it into partition files. Nested loops re-reads ENROLL's 50 blocks for each of the 600 students, about 30,000 block reads, yet was only 1.6 times slower in Experiment 1. ENROLL is only 20 KB, so those reads were most likely served from the operating system's file cache.

Join order in Experiment 2. Restricting to index joins also changed the join order. Plan A starts from DEPT (30 rows) and follows the two indexes, touching only matching rows. Plans B and C join DEPT and COURSE first, then STUDENT and ENROLL, so each step processes a larger intermediate result.

5. Declaration of AI Tool Use

We used one AI tool, Claude by Anthropic, through Claude Code with the Claude Sonnet 5 and Claude Opus 5.5 models. Table 4 lists how we used it in each part of the project. The rows for Labs 1 to 4 repeat the declarations in our lab reports.

Table 4: AI tool use by part of the project

Part of the project	How AI was used
Lab 1: non-equality predicates	To understand what each file did and to help our ideation, to generate test cases that we ran by hand in SimpleIJ, and to format the report and change list.
Lab 2: hash and B-tree indexes	To explore the existing index, catalog and planner code, and to help design how the index type is passed through the catalog. To generate IndexTypeTest and run the sample queries. To format the report and change list.
Lab 3: ORDER BY	To help write OrderByTest and draft the report. The engine changes were made without AI.
Lab 4: join algorithms	To help write JoinTest and CreateBigStudentDB and draft the report. The engine changes were made without AI.
Labs 5 to 7	We wrote the code. AI was used to explain concepts, discuss approaches, and debug code we had written.
Final integration	To help write two fixes found while testing: IndexJoinScan with an empty outer input, and SortPlan merging down to a single run. To help write JoinExperiments, the program that produced the timings in this report.
Report and change list	To draft and format this write-up and to consolidate the list of changes.
Final check	To test the integrated code against SQLite as a reference and to check this report's figures against the JoinExperiments output. No code from that check is part of this submission.